

CSRF

GRUPO 5

ABDIAS RUEDA
ALEXANDER CASTROVERDE
DONATHAN QUINTO
NADESH VALDES
WHITNEY AULT

¿QUE ES CSRF?

CSRF (CROSS-SITE REQUEST FORGERY) ES UN TIPO DE ATAQUE WEB QUE EXPLOTA LA CONFIANZA QUE UN SITIO TIENE EN EL NAVEGADOR DE UN USUARIO AUTENTICADO.

CUANDO TÚ INICIAS SESIÓN EN UNA PÁGINA (COMO UN BANCO, RED SOCIAL O CORREO), EL SISTEMA GUARDA UNA COOKIE DE SESIÓN EN TU NAVEGADOR PARA RECONOCER QUE ERES TÚ EN CADA SOLICITUD.



¿COMO FUNCIONA EL CSRF?

1. INICIO DE SESION

EL USUARIO ENTRA A UNA PÁGINA CONFIABLE (POR EJEMPLO, UN BANCO) E INICIA SESIÓN. EL SERVIDOR CREA UNA SESIÓN Y GUARDA UNA COOKIE DE AUTENTICACIÓN EN EL NAVEGADOR.

LA SESIÓN QUEDA ACTIVA MIENTRAS NO CIERRES SESIÓN. ESA COOKIE SIGUE ACTIVA. ESTO PERMITE QUE NO TENGAS QUE VOLVER A ESCRIBIR TU CONTRASEÑA EN CADA ACCIÓN.

2. ENLACE MALICIOSO

EL USUARIO ENTRA A OTRA WEB (PUEDE SER UN LINK, ANUNCIO O CORREO). ESTA PÁGINA CONTIENE CÓDIGO OCULTO PREPARADO POR EL ATACANTE. SE GENERA UNA SOLICITUD SIN QUE EL USUARIO LO NOTE LA PÁGINA MALICIOSA ENVÍA UNA PETICIÓN HACIA LA WEB ORIGINAL.

3. SE GENERA LA SOLICITUD

LA PÁGINA MALICIOSA ENVÍA UNA PETICIÓN HACIA LA WEB ORIGINAL.

EJEMPLO:

- TRANSFERIR DINERO
- CAMBIAR CONTRASEÑA
- ACTUALIZAR DATOS

TODO OCURRE EN SEGUNDO PLANO (SIN CLIC VISIBLE).

4. ENVÍA LA COOKIE

EL NAVEGADOR, SIN PREGUNTAR, ENVÍA LA COOKIE DE SESIÓN JUNTO CON LA SOLICITUD. EL SERVIDOR RECIBE:

- LA PETICIÓN
- LA COOKIE VÁLIDA

EL SERVIDOR PROCESA LA ACCIÓN COMO SI FUERA LEGÍTIMA.

MÉTODOS DE MITIGACIÓN



TOKENS ANTI-CSRF

VALORES ÚNICOS Y ALEATORIOS GENERADOS POR EL SERVIDOR QUE SE INCLUYEN EN CADA FORMULARIO O PETICIÓN.

EL SERVIDOR VALIDA ESTE TOKEN ANTES DE EJECUTAR CUALQUIER ACCIÓN PARA ASEGURARSE DE QUE LA SOLICITUD ES LEGÍTIMA.

POLÍTICA SAME-ORIGIN:

MECANISMO DE SEGURIDAD DEL NAVEGADOR QUE IMPIDE QUE SCRIPTS DE OTROS DOMINIOS INTERACTÚEN DIRECTAMENTE CON UN SITIO WEB.

AYUDA A EVITAR QUE PÁGINAS EXTERNAS ENVÍEN SOLICITUDES MALICIOSAS EN NOMBRE DEL USUARIO.

COOKIES SEGURAS SAMESITE:

CONFIGURACIÓN DE COOKIES QUE LIMITA SU ENVÍO EN SOLICITUDES ENTRE DIFERENTES SITIOS.

EN MODO STRICT O LAX, EVITA QUE LA COOKIE SE ENVÍE AUTOMÁTICAMENTE EN ATAQUES CSRF.

BUENAS PRÁCTICAS ADICIONALES:

CONJUNTO DE MEDIDAS COMPLEMENTARIAS PARA REDUCIR RIESGOS, COMO EVITAR USAR MÉTODOS GET EN ACCIONES CRÍTICAS Y APLICAR AUTENTICACIÓN MULTIFACTOR EN OPERACIONES SENSIBLES.

VULNERABILIDAD CSRF EN TIKTOK (2020)

INVESTIGADORES DESCUBRIERON UNA VULNERABILIDAD QUE PERMITÍA A UN ATACANTE CAMBIAR DATOS DE LA CUENTA, MANIPULAR CONFIGURACIONES DEL PERFIL E INCLUSO TOMAR CONTROL DE CUENTAS EN CIERTOS CASOS. TODO ESTO PODÍA LOGRARSE CON UN SOLO CLIC EN UN ENLACE MALICIOSO

¿CÓMO FUNCIONABA EL ATAQUE?

- EL USUARIO ESTABA AUTENTICADO EN TIKTOK.
- EL ATACANTE ENVIABA UN ENLACE MALICIOSO.
- AL HACER CLIC, SE ENVIABA UNA SOLICITUD FALSA AL SISTEMA.
- TIKTOK NO VALIDABA CORRECTAMENTE LA SOLICITUD.
- EL SISTEMA EJECUTABA LA ACCIÓN COMO SI FUERA EL USUARIO.

¿CÓMO SE SOLUCIONÓ?

TIKTOK CORRIGIÓ LA VULNERABILIDAD IMPLEMENTANDO:

- VALIDACIONES MÁS ERICTAS
- APLICANDO PROTECCIÓN CONTRA CSRF (TOKENS Y CONTROLES DE SEGURIDAD)



CASO PRACTICO

```
<?php
session_start();

// Simulación de login
$_SESSION['usuario'] = "cliente123";

// Acción crítica: transferencia de dinero
if (isset($_POST['monto']) && isset($_POST['destinatario'])) {
    $monto = $_POST['monto'];
    $destinatario = $_POST['destinatario'];

    // Aquí se ejecuta la transferencia sin validación adicional
    echo "Transferencia de $monto realizada a $destinatario";
}
?>

<!-- Formulario vulnerable -->
<form method="POST" action="transferencia.php">
    Monto: <input type="text" name="monto"><br>
    Destinatario: <input type="text" name="destinatario"><br>
    <input type="submit" value="Transferir">
</form>
```

- EL FORMULARIO NO INCLUYE UN TOKEN ANTI-CSRF.
- EL SERVIDOR ACEPTA CUALQUIER PETICIÓN POST SIEMPRE QUE EL USUARIO ESTÉ AUTENTICADO.
- UN ATACANTE PUEDE CREAR UN FORMULARIO OCULTO EN OTRA PÁGINA QUE ENVÍE DATOS A TRANSFERENCIA.PHP.
- SI EL USUARIO ESTÁ LOGUEADO EN EL SISTEMA Y VISITA LA PÁGINA MALICIOSA, EL NAVEGADOR ENVIARÁ AUTOMÁTICAMENTE LAS COOKIES DE SESIÓN, EJECUTANDO LA TRANSFERENCIA SIN QUE EL USUARIO LO NOTE.

EJEMPLO DE ATAQUE

```
<form method="POST" action="http://victima.com/transferencia.php" style="display:none">
  <input type="hidden" name="monto" value="1000">
  <input type="hidden" name="destinatario" value="atacante123">
  <input type="submit">
</form>

<script>
  document.forms[0].submit();
</script>
```

UN ATACANTE PODRÍA INCRUSTAR ESTE CÓDIGO EN SU SITIO. CUANDO EL USUARIO AUTENTICADO VISITE ESA PÁGINA, SE EJECUTARÁ LA TRANSFERENCIA AUTOMÁTICAMENTE. ESTE EJEMPLO MUESTRA CÓMO LA AUSENCIA DE TOKENS ANTI-CSRF Y VALIDACIONES DE ORIGEN DEJA EXPUESTA LA APLICACIÓN.

CORRECCIÓN DE ESTE TIPO DE ATAQUE.

```
<?php
session_start();

// Simulación de login
$_SESSION['usuario'] = "cliente123";

// Generar token CSRF único por sesión
if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// Acción crítica: transferencia de dinero
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
        die("Error: Token CSRF inválido.");
    }

    $monto = $_POST['monto'];
    $destinatario = $_POST['destinatario'];

    echo "Transferencia de $monto realizada a $destinatario";
}

?>

<!-- Formulario protegido -->
<form method="POST" action="transferencia.php">
    Monto: <input type="text" name="monto"><br>
    Destinatario: <input type="text" name="destinatario"><br>
    <!-- Token oculto -->
    <input type="hidden" name="csrf_token" value="<?php echo $_SESSION['csrf_token']" />
    <input type="submit" value="Transferir">
</form>
```

- SE GENERA UN TOKEN ALEATORIO (BIN2HEX(RANDOM_BYTES(32))) Y SE GUARDA EN LA SESIÓN.
- EL FORMULARIO INCLUYE ESE TOKEN EN UN CAMPO OCULTO.
- EL SERVIDOR VALIDA QUE EL TOKEN RECIBIDO COINCIDA CON EL DE LA SESIÓN ANTES DE EJECUTAR LA ACCIÓN.
- SI EL TOKEN NO EXISTE O NO COINCIDE, LA PETICIÓN SE RECHAZA.

GRACIAS